SDS 2206

Data Visualization Analytics and Reporting

PROJECT REPORT Milestones 1 & 2

System	Sports Performance Analytics Visualization System
Subject	SDS 2206 – Data Visualization Analytics and Reporting
Milestones	Milestone 1 (Weeks 1–2) & Milestone 2 (Weeks 3–4)
Dataset	esports dataset.csv (structured tabular data)
Tools	Python 3, Pandas, Matplotlib, NumPy

1. Introduction

This report documents the development of a Sports Performance Analytics Visualization System, built on an esports player and match dataset. The project follows a six-milestone program under the Unit SDS 2206: Data Visualization Analytics and Reporting, with each milestone evolving the system progressively from raw data representation through to research-level visual analytics.

Milestone 1 establishes the data foundation acquiring, describing, and initially exploring the dataset through basic statistics and simple visualizations. Milestone 2 builds upon this foundation by constructing a robust, reproducible data processing pipeline that cleans, validates, transforms, and enriches the dataset with engineered features, making it ready for advanced visualization and modelling in subsequent milestones.

The system focuses on esports performance data, a domain that generates rich structured data across player roles, team compositions, match events, and physiological indicators such as fatigue and reaction time. Extracting actionable insights from this data requires a systematic pipeline the construction of which is the central goal of these first two milestones.

2. Problem Definition

Modern esports competitions generate large volumes of granular player and match data. While this data holds significant potential for performance analysis and strategic decision-making, several structural challenges must be overcome:

- Inconsistent data formats: Raw exported data frequently mixes types, uses varied conventions, and contains structural irregularities.
- Missing and noisy values: Player records may be incomplete due to disconnections, tracking failures, or data entry errors.
- Out-of-range measurements: Numerical values such as accuracy percentages or reaction times may fall outside physiologically plausible bounds.
- Lack of composite metrics: Raw columns like kills, assists, and deaths do not alone capture performance derived features are required.

This system addresses these challenges by building an end-to-end analytics pipeline. The specific objectives for Milestones 1 and 2 are:

- Acquire and describe the dataset structure and variable types.
- Compute foundational descriptive statistics and perform an initial quality assessment.
- Design and implement a reproducible data cleaning and transformation pipeline.
- Engineer meaningful performance features to enhance the analytical richness of the dataset.
- Produce an analysis-ready, cleaned dataset for downstream visualization tasks.

3. Milestone 1: Data Representation & Foundations

Milestone 1 establishes the analytical foundation of the system by acquiring the dataset, describing its structure, computing basic statistics, generating initial visualizations, and identifying data quality concerns that inform the Milestone 2 cleaning pipeline.

3.1 Dataset Acquisition

The dataset was sourced as a structured CSV file (esports dataset.csv) and stored locally under Data/Raw/. It was loaded into the Python environment using the Pandas library. Upon loading, initial inspection was performed using df. shape, df. dtypes, df.isna().sum(), and df.head() to understand the dataset's dimensions, column types, missing value distribution, and sample records.

Source	CSV file esports dataset.csv
Storage Path	Data/Raw/esports dataset.csv
Load Method	pandas.read_csv() with default settings

3.2 Data Schema & Variable Definitions

The dataset contains both categorical and numerical variables capturing player identity, match context, and in-game performance metrics.

3.2.1 Categorical Variables

Variable	Description
team_name	The esports team the player belongs to
player_role	In-game role (Entry Fragger, Sniper, IGL, Support, Flex)
map_played	The map on which the match was played
match_type	Type of match (competitive, practice, etc.)
match_outcome	Result of the match from the team's perspective
mvp_award	Whether the player received the MVP award for the match

3.2.2 Numerical Variables

Variable	Unit	Description
kills	Count	Number of enemy eliminations by the player
assists	Count	Number of assists on enemy eliminations

deaths	Count	Number of times the player was eliminated
accuracy_percent	%	Shot accuracy as a percentage (0–100)
reaction_time_ms	ms	Measured player reaction time (80–600 ms)
fatigue_index	0–1 scale	Composite fatigue measure (0 = fresh, 1 = maximum fatigue)
performance_score	Score	Composite performance score computed by the system
win_probability	0–1 scale	Modelled probability of the team winning the match

3.3 Basic Descriptive Statistics

Using Pandas' df.describe(include='all') method, comprehensive descriptive statistics were computed across all columns. Key metrics examined included central tendency (mean and median), spread (standard deviation and variance via df.var(numeric_only=True)), range (minimum and maximum values), and frequency counts for categorical variables.

These statistics provided the first structured view of data distribution and highlighted anomalous areas particularly in accuracy_percent and win_probability, where extreme values were identified. The kills distribution showed a roughly bell-shaped pattern centered around 13–17 kills per match. Fatigue index appeared approximately uniformly distributed across [0, 1], while performance scores followed a near-normal distribution centered around 45–50.

3.4 Initial Visual Outputs

A set of basic visualizations was generated to support initial exploratory analysis, providing early evidence of data distributions, categorical compositions, and variable relationships.

Figure 1: Kills Distribution and MVP Award Counts

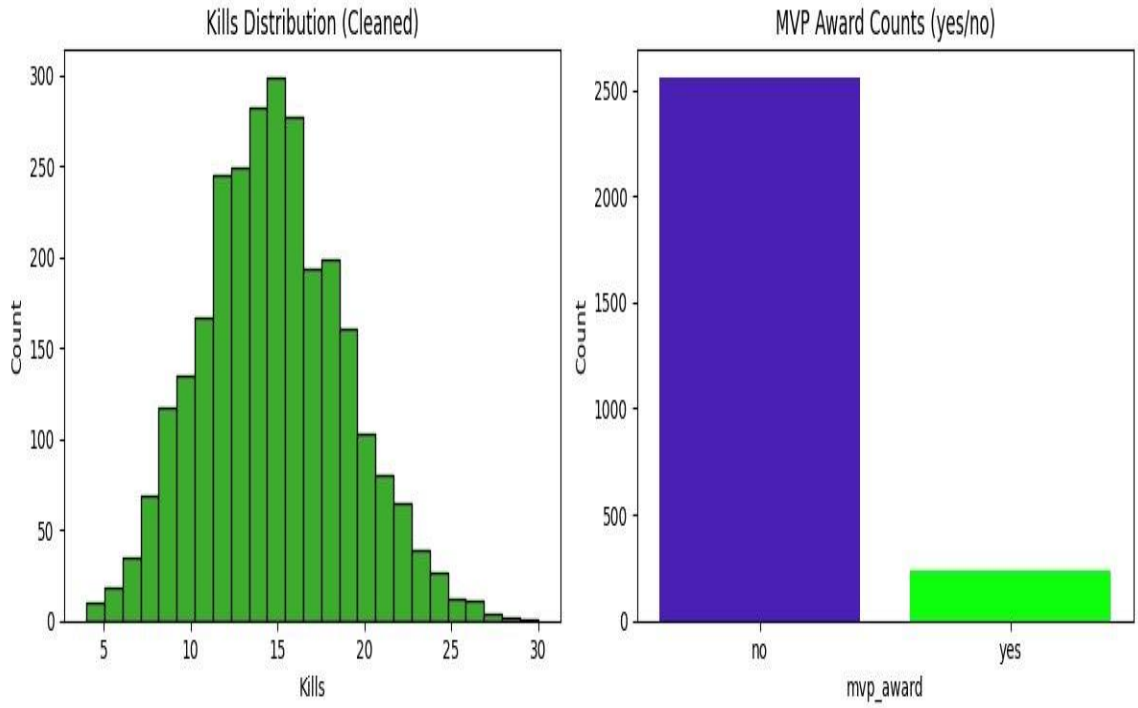

Figure 1: Kills histogram (left) showing a bell-shaped distribution peaking at 13–17 kills. MVP award bar chart (right) showing a strong class imbalance between non-recipients and recipients.

The kills histogram reveals a clean approximately normal distribution, consistent with expected esports performance ranges. The MVP bar chart highlights a significant class imbalance the vast majority of players did not receive MVP, with 'yes' representing a small minority relevant to future classification modelling.

Figure 2: Accuracy vs. Win Probability

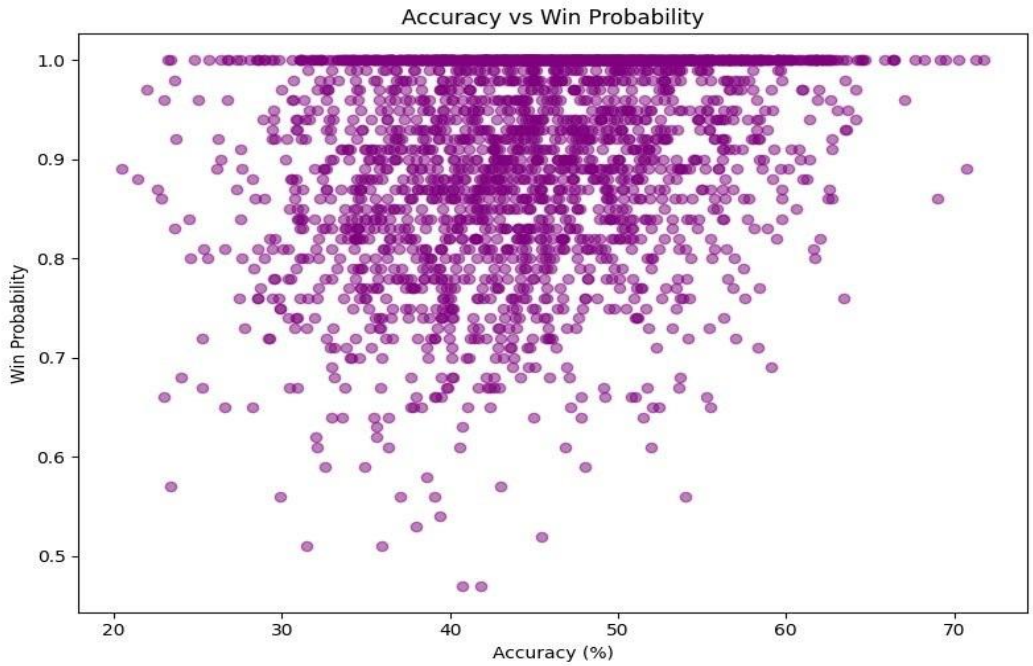

Figure 2: Scatter plot of accuracy (%) vs. win probability. High density at win probability = 1.0 indicates a ceiling effect; no strong linear relationship is observable.

The scatter plot reveals a notable ceiling effect: a substantial proportion of observations cluster at win probability = 1.0 regardless of accuracy level. This suggests win probability may function as a near-binary variable rather than a continuous probabilistic measure, which will require careful handling in downstream analysis.

3.5 Data Quality Assessment

A systematic data quality assessment was conducted, revealing three categories of issues:

Issue Category	Observation	Impact
Missing Values	Multiple numerical and categorical columns contained NaN entries	Breaks statistical calculations and model training
Inconsistent Labels	Categorical fields used mixed case, whitespace, and variant spellings (e.g., 'Yes', 'yes', 'Y')	Causes incorrect grouping and frequency counts

Out-of-Range Values	Numerical fields contained values outside plausible real-world bounds	Distorts descriptive statistics and visualizations

This quality assessment directly drove the design of the Milestone 2 cleaning pipeline.

3.6 Milestone 1 Deliverables Summary

Deliverable	Status
Working Python prototype (main.py)	Completed
Dataset description and schema	Completed
Basic descriptive statistics	Completed
Initial visual outputs (kills, MVP, accuracy vs win probability)	Completed
Data quality assessment report	Completed

4. Milestone 2: Data Processing & Transformation

Milestone 2 transforms the raw dataset from Milestone 1 into a clean, structured, and analytically enriched dataset. A fully reproducible Python pipeline was implemented, encompassing data cleaning, validation, transformation, and feature engineering.

4.1 Data Cleaning Pipeline

4.1.1 Duplicate Removal

Exact duplicate rows were identified and removed using Pandas' drop_duplicates() method. Retaining duplicates would artificially inflate frequency counts and distort distributional statistics.

4.1.2 String Standardization

All categorical columns were standardized through the following sequential operations: convert to string type, strip leading/trailing whitespace, convert to lowercase, and replace placeholder strings ('', 'none', 'null', 'nan') with proper NaN values. This ensures that labels like 'Yes', 'yes', and 'YES' are treated as identical.

4.1.3 Numeric Type Coercion

All numerical columns were converted using pd.to_numeric() with errors='coerce', which converts non-numeric entries to NaN rather than raising an exception, safely handling mixed-type columns without data loss.

4.1.4 Missing Value Imputation

Following type coercion, residual missing values in numerical columns were filled with a placeholder value of 1 chosen as a neutral, non-zero default that does not distort ratio-based features such as KDA. Categorical missing values were retained as NaN to preserve their informativeness.

4.2 Data Validation & Anomaly Handling

Each numerical feature was validated against defined plausible real-world bounds. Values falling outside these ranges were set to NaN, preventing anomalous measurements from distorting aggregate statistics.

Feature	Valid Range	Rationale
accuracy_percent	0 – 100 (%)	A percentage cannot be negative or exceed 100
win_probability	0.0 – 1.0	A probability must fall within [0, 1]

fatigue_index	0.0 – 1.0	Normalized index bounded to [0, 1]
reaction_time_ms	80 – 600 (ms)	Human reaction times fall within a physiologically plausible window
kills	0 – 80	Match-level kill counts are bounded by game mechanics
assists	0 – 80	Match-level assist counts are bounded by game mechanics
deaths	0 – 80	Match-level death counts are bounded by game mechanics

4.3 Data Transformation & Aggregation

4.3.1 Team-Level Aggregation

Player-level records were aggregated at the team level to compute total kills, assists, and deaths per team using Pandas' groupby() operation, providing a team performance view that complements the individual player perspective.

4.3.2 Role and Map Distribution

Value counts were computed for player_role and map_played to understand the distribution of observations across roles and maps, informing whether the dataset is balanced across categories relevant for future comparative analysis.

4.4 Feature Engineering

Three new features were engineered to enhance the analytical depth of the dataset:

Feature	Formula	Purpose
kda_ratio	(kills + assists) / deaths	Composite performance measure balancing offensive contribution and survivability
kill_death_diff	kills − deaths	Net combat efficiency indicator positive values mean more eliminations than deaths
is_mvp	1 if mvp_award == 'yes'	Binary numeric encoding of MVP award status for statistical and predictive models

Deaths of zero were handled by substituting NaN as the denominator in the KDA calculation to prevent division-by-zero errors. The mvp_award field was standardized (mapping 'y', 'true', '1' → 'yes' and 'n', 'false', '0' → 'no') before binary encoding.

4.5 Visualization Outputs from the Cleaned Dataset

After cleaning and feature engineering, a comprehensive visualization suite was generated to validate the pipeline and surface early insights.

Figure 3: Performance Score Distribution

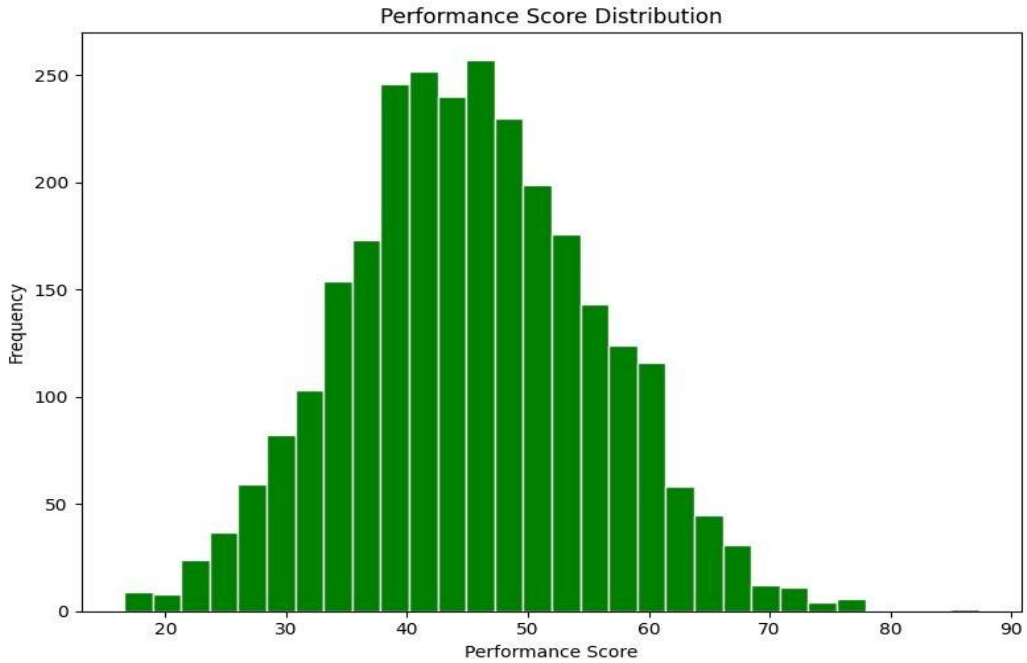

Figure 3: Histogram of performance scores after cleaning. The near-normal distribution, peaking between 43–48, validates that the pipeline has not introduced systematic bias.

The performance score distribution follows a near-normal curve, peaking between 43 and 48. This bell-shaped pattern is consistent with expectations for a balanced competitive dataset and confirms the cleaning pipeline produced well-distributed output. The right tail (scores above 65) represents high-performing outliers of analytical interest.

Figure 4: Fatigue Index Distribution

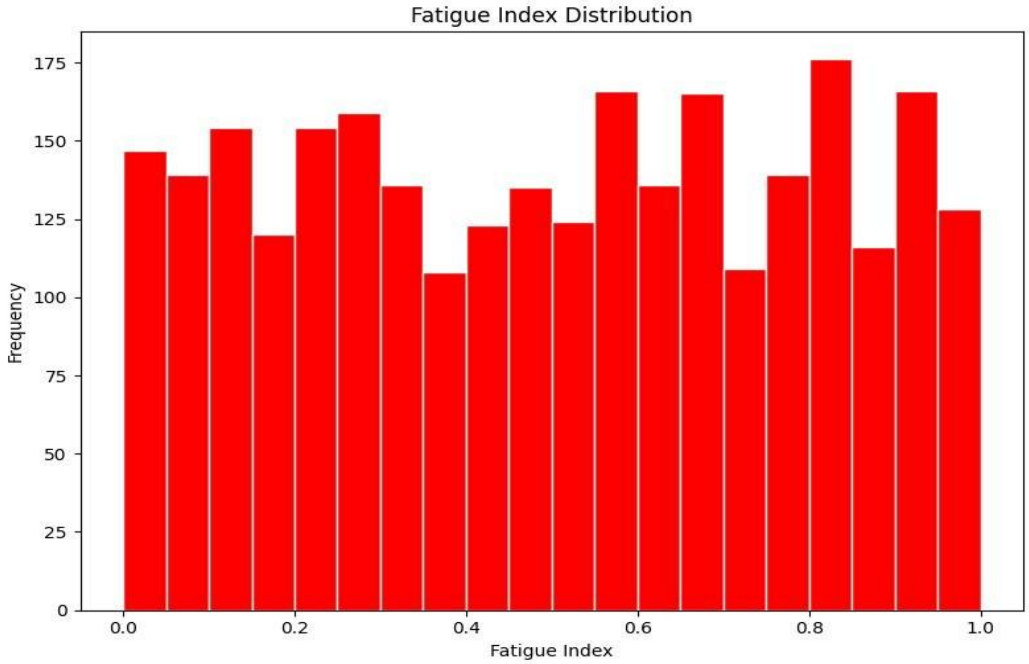

Figure 4: Histogram of the fatigue index. The approximately uniform distribution across [0, 1] confirms good coverage of fatigue conditions in the dataset.

The fatigue index is approximately uniformly distributed across the full [0, 1] range, indicating the dataset captures performance across diverse fatigue states from well-rested to maximally fatigued. This is valuable for analysing the relationship between fatigue and performance outcomes in later milestones.

Figure 5: Fatigue Index vs. Performance Score

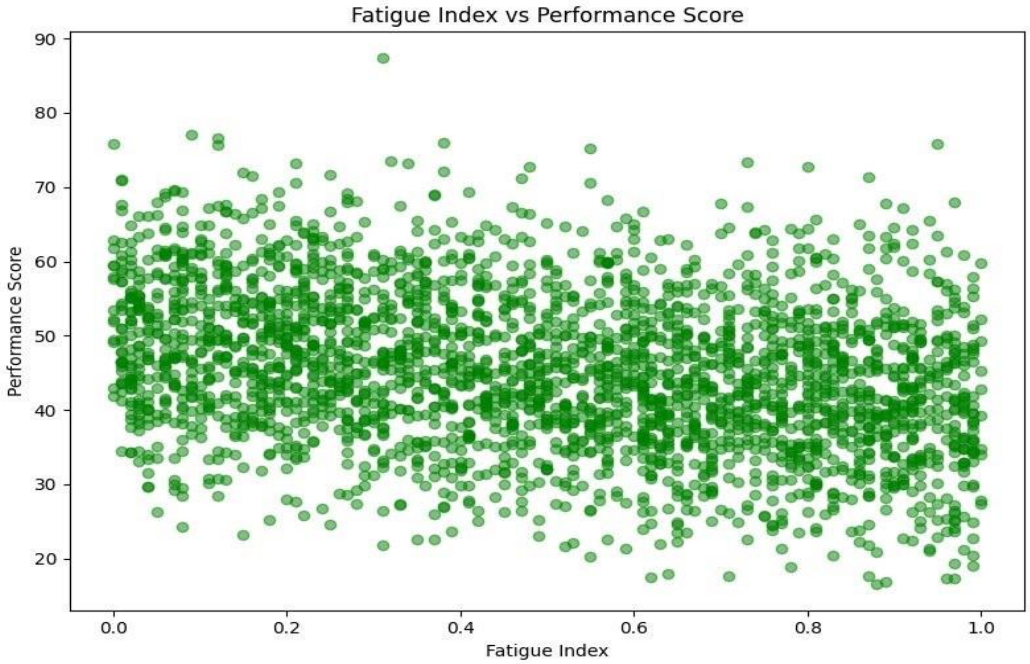

Figure 5: Scatter plot of fatigue index vs. performance score. No strong linear relationship is visible, suggesting fatigue's effect on performance is non-linear or mediated by other variables.

The scatter plot reveals no strong linear correlation between fatigue index and performance score performance is broadly distributed across all fatigue levels. This finding motivates multi-variable analysis in later milestones, potentially including interaction terms or non-linear models.

Figure 6: Reaction Time Distribution

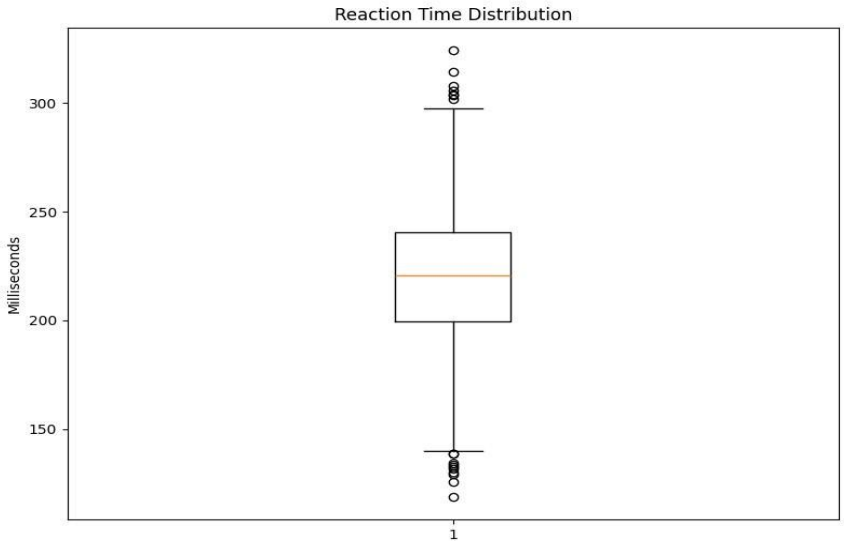

Figure 6: Box plot of reaction time (ms). Median ~220 ms, IQR from ~190 to ~245 ms, with a small number of outliers above 300 ms.

Player reaction times are concentrated between approximately 190 ms (Q1) and 245 ms (Q3), with a median near 220 ms. Outliers above 300 ms represent players whose reaction times fall substantially outside the typical range potentially indicating fatigue, technical issues, or genuine performance variation worth investigating.

Figure 7: Distribution of Player Roles

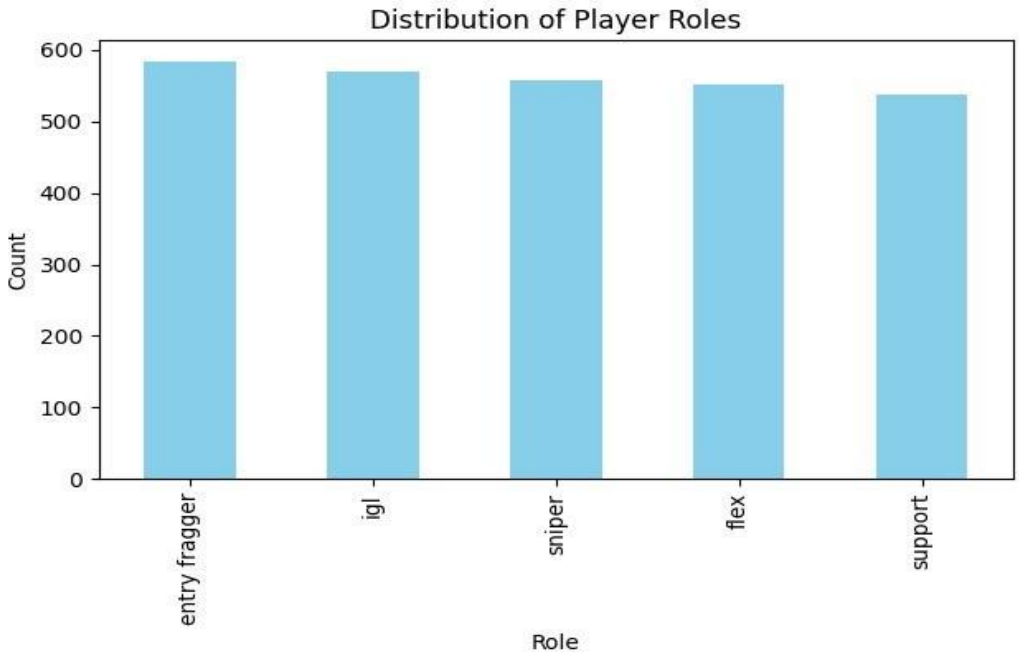

Figure 7: Bar chart of player role counts. Approximately balanced distribution across Entry Fragger, IGL, Sniper, Flex, and Support roles (540–585 each).

Player roles are distributed approximately evenly across five categories, with counts ranging from ~540 to ~585. This near-uniform distribution confirms the dataset does not over-represent any single role, which is important for valid role-based comparisons in subsequent milestones.

Figure 8: Distribution of Maps Played

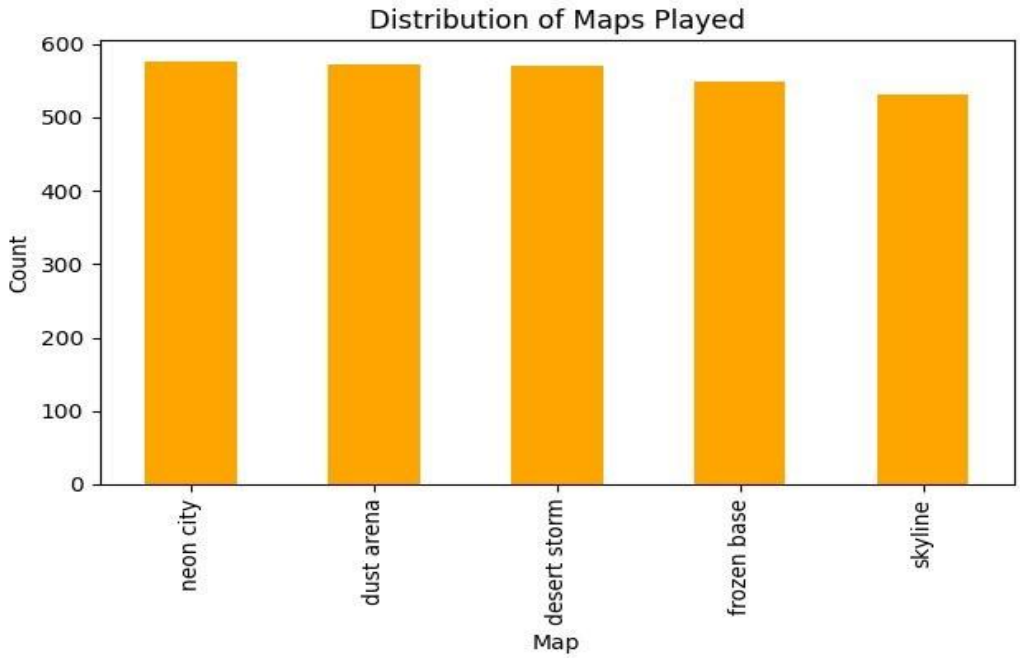

Figure 8: Bar chart of match counts across five maps. All maps are well-represented with counts between 530 and 580.

The five maps Neon City, Dust Arena, Desert Storm, Frozen Base, and Skyline are each represented by 530–580 matches. This balanced distribution allows fair map-to-map comparisons and avoids bias in map-specific analyses.

Figure 9: Team Kills, Assists, and Deaths

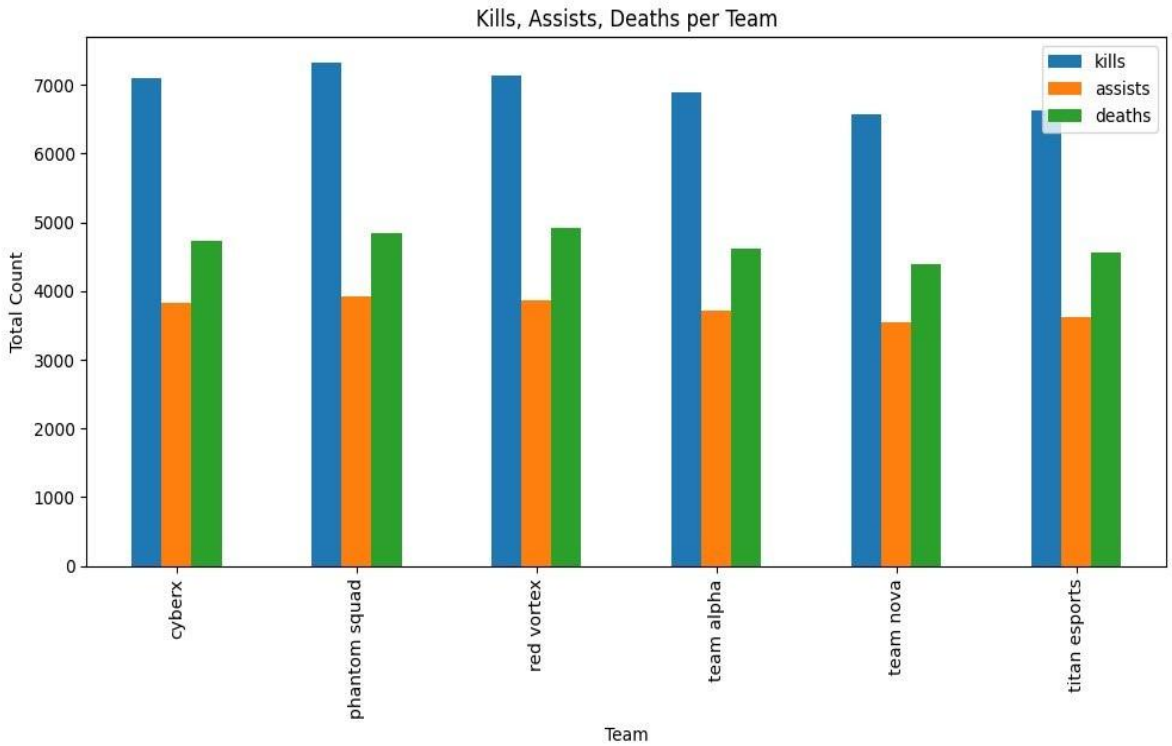

Figure 9: Grouped bar chart of total kills, assists, and deaths per team. All six teams show broadly comparable aggregate combat statistics, with Phantom Squad leading in total kills (~7,300).

Total kills, assists, and deaths are broadly comparable across all six teams CyberX, Phantom Squad, Red Vortex, Team Alpha, Team Nova, and Titan Esports. Kills consistently outnumber deaths across all teams, and assists are approximately half the kill count. Phantom Squad leads in total kills (~7,300) while Team Nova records the fewest (~6,600), marginal differences worth investigating in later milestones.

4.6 Pipeline Implementation & Reproducibility

The complete data pipeline is implemented across two Python scripts:

Script	Purpose
main.py	End-to-end pipeline: loading, cleaning, validation, feature engineering, saving cleaned CSV
data_analysis.py	Post-cleaning visualization and descriptive analysis on the processed dataset

The pipeline follows a strictly sequential structure, ensuring every step is documented and reproducible:

- Step 1: Load raw CSV from Data/Raw/
- Step 2: Remove exact duplicate rows
- Step 3: Standardize string columns (strip, lowercase, replace placeholders)
- Step 4: Coerce numerical types with errors='coerce'
- Step 5: Apply range validation; flag out-of-bound values as NaN
- Step 6: Impute missing numerical values with 1
- Step 7: Engineer kda_ratio, kill_death_diff, and is_mvp features
- Step 8: Save cleaned dataset to data/processed/cleaned_esports_data.csv
- Step 9: Generate and save visualization outputs to visuals/

4.7 What Changed from Milestone 1

Dimension	Milestone 1	Milestone 2
Data State	Raw, unprocessed CSV	Cleaned, validated, type-consistent
Missing Values	Present and unaddressed	Imputed (numerical) or preserved (categorical)
Category Labels	Mixed case, whitespace, variants	Standardized: lowercase, stripped, mapped
Numeric Bounds	Not validated	Range-validated; out-of-bound values → NaN
Features	Original columns only	3 new features: kda_ratio, kill_death_diff, is_mvp
Pipeline	Ad hoc analysis script	Fully reproducible, sequential pipeline
Output	Console output + basic charts	Cleaned CSV + 9-chart visualization suite

4.8 Issues Encountered & Solutions Applied

Issue 1 Missing Value Strategy

Initial Approach	Leave missing values as NaN without intervention
Problem	Statistical calculations (mean, standard deviation, KDA ratio) returned NaN or failed when missing values propagated

Solution	Replaced all numerical NaN values with 1 after type coercion and range validation. Value of 1 was chosen as a neutral, non-distorting placeholder that preserves ratio calculations.

Issue 2 Inconsistent Category Labels

Initial Approach	Process categorical fields without normalization
Problem	Value counts produced fragmented results 'Yes', 'yes', and 'Y' were counted as three separate categories, distorting frequency analysis
Solution	Applied a sequential standardization chain (strip → lowercase → map variants) to unify all label formats before analysis

Issue 3 Invalid Numerical Ranges

Initial Approach	Accept all numerical values at face value
Problem	Values outside plausible ranges (e.g., accuracy > 100%, win probability > 1.0) distorted descriptive statistics and scatter plot patterns
Solution	Implemented per-column range validation rules; out-of-bound values set to NaN before imputation, ensuring they did not contaminate summary statistics

4.9 Milestone 2 Deliverables Summary

Deliverable	Status
Clean and structured dataset (cleaned_esports_data.csv)	Completed
Reproducible data pipeline (main.py)	Completed
Feature-engineered dataset (kda_ratio, kill_death_diff, is_mvp)	Completed
Visualization suite (9 charts saved to visuals/)	Completed
Transformation documentation (this report)	Completed

5. Conclusion

Milestones 1 and 2 have jointly established a solid, reproducible foundation for the Sports Performance Analytics Visualization System. The key accomplishments across both milestones are:

- A well-described dataset with clearly defined variable types and a documented schema.
- A systematic data quality assessment that identified and classified three categories of issues:missing values, inconsistent labels, and out-of-range measurements.
- A fully reproducible Python pipeline that cleans, validates, transforms, and enriches the rawdataset.
- Three engineered features KDA ratio, kill-death difference, and is_mvp that enhance thedataset's analytical richness.

- A comprehensive 9-chart visualization suite confirming that the pipeline produces well-distributed,consistent output data.

The cleaned dataset (data/processed/cleaned_esports_data.csv) is now analysis-ready and will serve as the input for all subsequent milestones. The pipeline is fully reproducible, documented, and extensible ensuring future modifications can be traced and validated.

6. Next Steps Milestone 3 Preview

Milestone 3 will focus on Visualization and Exploratory Analysis, building on the cleaned dataset to produce a richer, design-principled visual analytics suite. Planned work includes:

- Multiple chart types: line charts for temporal trends, scatter plots with regression overlays, and heatmaps for correlation matrices.
- Application of visualization design principles: color theory, perceptual hierarchy, data-ink ratio,and accessibility.
- Comparative visualization strategies: side-by-side comparisons of player roles, map performance and team compositions.
- Insight-driven storytelling: narrative framing of key findings to support decision-making.
- Design justification documentation: formal rationale for chart type selections and design choices.

The system will continue to evolve progressively, with each milestone building directly upon the outputs of its predecessors.